Jev: Khi AI thôi viết văn và bắt đầu ra quyết định

System One Model: AI không sinh chữ, chỉ trả về quyết định có kiểu kèm xác suất



MỞ ĐẦU

Bạn đã bao giờ gọi GPT chỉ để lấy về đúng một chữ “có” hoặc “không”?

LLM trả lời

“Dựa trên nội dung tin nhắn và lịch sử của khách hàng, tôi cho rằng yêu cầu này nên được chuyển cho bộ phận kỹ thuật, vì...”

Rồi bạn phải viết thêm cả một chương trình

Prompt “chỉ trả JSON” → parse → validate → gỡ markdown fence → retry khi enum sai → fallback khi model xin lỗi thay vì trả lời

Trong khi code chỉ cần

```
team = "technical"
```

Phần lớn lời gọi AI trong phần mềm là để **phán đoán**, không phải để **viết**.
Vậy mà ta vẫn đang trả chi phí của việc viết để nhận được phán đoán.

Gọi Jev giống như gọi một hàm trong code

Mô hình đầu tiên của TypeSafe AI thuộc lớp **System One Model**: nhận trạng thái chương trình cùng các câu hỏi có kiểu, trả về quyết định có kiểu kèm xác suất. **Không sinh văn bản.**

Vào

Dữ liệu phi cấu trúc (tin nhắn, JSON, bản ghi) + câu hỏi do bạn định nghĩa

→

Jev

Một lượt tính song song cho mọi câu hỏi cùng lúc

→

Ra

Giá trị có kiểu: lựa chọn, điểm, xác suất, kèm độ tin cậy

70–500 ms

mỗi lần gọi

0,042 USD

cho 1 triệu token đầu vào

0 USD

token đầu ra (vì không sinh token nào)

PHẦN 1 · NGUỒN GỐC

Người dạy AI trò chuyện với con người, nay làm AI chỉ nói chuyện với phần mềm

Diogo Almeida · CEO

Từng ở OpenAI, góp phần xây phương pháp InstructGPT và RLHF, nền móng nghiên cứu phía sau ChatGPT.

Đồng sáng lập: **Sasha Sheng** (COO) và **Erik Gafni** (CTO).

CÂU HỎI DẪN DẮT

AI trò chuyện đã giỏi hơn con người nhiều năm nay. Vậy sao các doanh nghiệp vẫn chưa tự động hóa được bao nhiêu việc?

2024

Rời OpenAI, lập TypeSafe AI tại San Francisco

2 năm cày trong im lặng

Kiến trúc mới, bộ sampler song song, phương pháp huấn luyện RLCD

15/9/2026

Ra mắt Jev (jev-1.13.0, early access) · seed 40 triệu USD do DCVC dẫn dắt

PHẦN 1 · CÁI TÊN

Hai cái tên, một luận điểm

System One

Daniel Kahneman · "Thinking, Fast and Slow"

Hệ 1: nhanh, trực giác. Nhìn khuôn mặt giận dữ là biết ngay.

Hệ 2: chậm, có chủ đích. Nhẩm 17 × 28 thì phải dừng lại tính.

LLM hôm nay vận hành kiểu Hệ 2, trong khi phần lớn quyết định trong phần mềm lại là Hệ 1.

Jev

Đặt theo tên William Stanley Jevons, nhà kinh tế học người Anh

Năm 1865: máy hơi nước được cải tiến, tốn ít than hơn. Ai cũng nghĩ than sẽ được dùng ít đi.

Thực tế: chạy máy rẻ nên nhà máy mọc khắp nơi, tổng lượng than dùng lại tăng vọt.

Với AI: mỗi lần ra quyết định rẻ đi khoảng 200 lần, AI sẽ được gắn vào hàng nghìn chỗ trước đây không đáng tiền.

PHẦN 1 · CÁCH ĐẶT CÂU HỎI CHO JEV

Hỏi Jev giống như phát một phiếu trắc nghiệm

① State

Thứ cần phán đoán: văn bản, JSON, bản ghi

"Tôi thử kết nối tài khoản Stripe 3 ngày nay mà vẫn lỗi. Tôi đang mất doanh thu. Giúp tôi gấp!"

② Questions

Bạn soạn phiếu, kể cả đáp án được phép

Bộ phận nào xử lý?
billing / technical / sales

Khách bực đến mức nào?
thang 3 mức

Tin nhắn có gấp không?
đúng / sai

③ Answers

Jev điền phiếu trong một lượt, khoảng 400 ms

department
→ "technical"

frustration
→ 1.035

is_urgent
→ 0.999

Jev **không thể** viết thêm đáp án ngoài phiếu: không có bộ phận thứ tư nào được bịa ra.

Code của bạn đọc kết quả và quyết định bước tiếp theo, không phải model.

Jev chỉ hiểu 3 loại câu hỏi, và chỉ có 3

CHOICE

Chọn 1 trong N

Dùng khi: phân loại, định tuyến, chọn hành động tiếp theo

Bạn khai báo: tối đa 255 phương án, mỗi phương án kèm mô tả

Jev trả về: phương án thắng, xác suất của từng phương án, độ tin cậy

billing 0.91
technical 0.06
account 0.03

SCORE

Chấm trên thang

Dùng khi: mức nghiêm trọng, cảm xúc, chất lượng, độ ưu tiên

Bạn khai báo: 2 đến 10 mức có thứ tự, mô tả mỗi mức bằng lời

Jev trả về: một vị trí trên thang (có thể lẻ), độ tin cậy

0 Bình tĩnh
1 Bực nhưng lịch sự
2 Rất giận → 1.035

NOUL

Đúng hay sai?

Dùng khi: kiểm tra điều kiện, guardrail, "xong chưa?", "có liên quan không?"

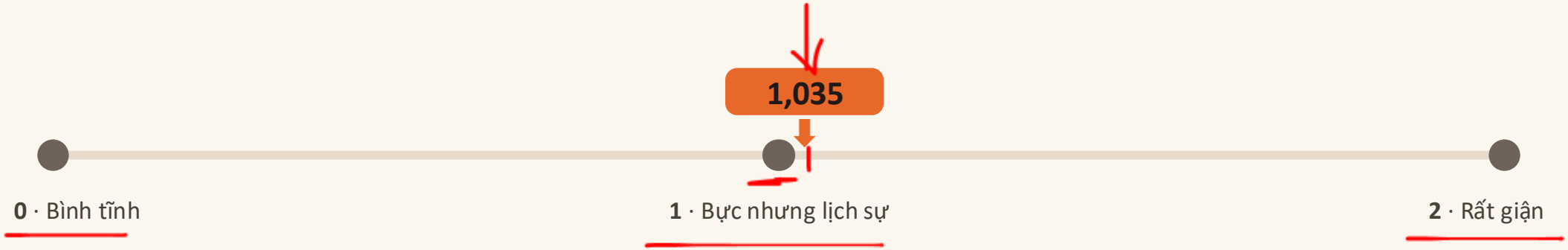
Bạn khai báo: một mệnh đề khẳng định

Jev trả về: xác suất mệnh đề đó đúng, từ 0 đến 1

"Tin nhắn thể hiện
sự gấp gáp"
→ 0.999

PHẦN 1 · ĐỌC KẾT QUẢ SCORE

Score trả về một vị trí trên thang, không phải một nhãn



Choice và Score khác nhau thế nào?

Choice chọn một nhãn:
"Bực nhưng lịch sự".

Score chấm một điểm trên thước:
1,035, tức là vừa nhích qua mức 1.

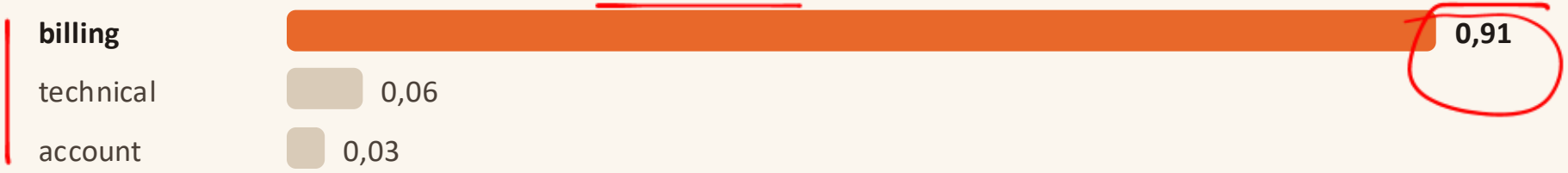
Vì sao cần số lẻ?

Số càng gần mức nào, khách càng giống mức đó.

Phân biệt được "vừa chớm bực" (1,0) với "sắp nổi nóng" (1,8), dù cả hai đều thuộc mức 1.

Choice trả về một lựa chọn, kèm xác suất của từng phương án

Tin nhắn: “Tôi bị trừ tiền hai lần cho cùng một gói đăng ký.” · **Câu hỏi:** Đội nào xử lý?



Đọc kết quả

choice = “billing”: phương án có xác suất cao nhất.

Xác suất các phương án cộng lại bằng 100%.

Kết quả luôn nằm trong danh sách bạn khai báo.

Dùng trong code

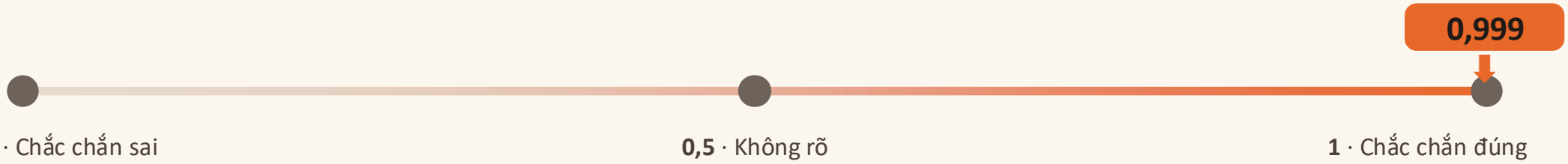
Thắng rõ (0,91): tự động chuyển cho billing.

Sát nút (vd 0,52 so với 0,46): chuyển người duyệt.

Trường **confidence** cho biết mức “chắc” của cả phân phối.

PHẦN 1 · ĐỌC KẾT QUẢ NOUL

Noul trả về xác suất một câu khẳng định là đúng



Cách hỏi

Viết một **câu khẳng định**, không phải câu hỏi: “Tin nhắn thể hiện sự gấp gáp”.

Jev trả về xác suất câu đó đúng, từ 0 đến 1.

Đọc con số

0,999: gần như chắc chắn tin nhắn này gấp.

Đây **không phải** “mức độ gấp”. Muốn đo mức độ, dùng Score.

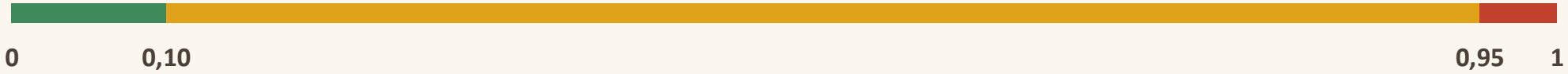
Trong code: chọn ngưỡng, vd trên 0,7 thì gắn cờ gấp.

Kết quả thật của ticket Stripe ở slide 6

PHẦN 1 · ĐẶT NGƯỠNG

Model chấm điểm, code bật đèn giao thông

Noul: “Tin nhắn này là lừa đảo giả mạo ngân hàng” → Jev trả về xác suất từ 0 đến 1



Dưới 0,10 · Cho qua

Gần như chắc chắn an toàn.

Tin nhắn đi thẳng tới khách, không ai phải xem.

0,10 – 0,95 · Người duyệt

Không đủ chắc để tự quyết.

Gắn cảnh báo cho khách, hoặc chuyển nhân viên kiểm tra.

Trên 0,95 · Tự chặn

Đủ chắc để hành động ngay.

Chặn tin nhắn, không cần người duyệt.

Nguyên tắc: ngưỡng do người đặt, nằm trong code

Sai càng đắt, ngưỡng càng chặt: xem số dư có thể tự làm ở 0,7; chuyển tiền phải trên 0,95, còn lại hỏi lại khách. Đổi ngưỡng là đổi tỷ lệ tự động hóa, không cần huấn luyện lại model.

Một lần gọi, ba câu trả lời

```
res = client.system_one(state=ticket, questions={
  "department": Choice(instructions="Bộ phận nào xử lý ticket này?",
    criteria={"billing": "Thanh toán",
      "technical": "Kỹ thuật",
      "sales": "Kinh doanh"}),
  "frustration": Score(instructions="Bực cỡ nào?",
    criteria=["Bình tĩnh", "Bực, lịch sự", "Rất giận"]),
  "is_urgent": Noul(
    instructions="Tin nhắn thể hiện sự gấp gáp"),
})

res.answers["department"].choice  # "technical"
res.answers["frustration"].score  # 1.035
res.answers["is_urgent"].noul     # 0.999
```

≈ 400 ms

1 request, 3 phán đoán, 1 vòng mạng

Không prompt dặn “chỉ trả JSON”: định dạng không do model chọn.

Không json.loads bọc trong try/except.

Có phân phối xác suất và độ tin cậy để code tự quyết định.

Rút gọn từ ví dụ Python SDK (typesafe-sdk); mô tả đã dịch sang tiếng Việt

Ra mắt chưa đầy một tuần, cộng đồng đã dựng đủ thứ

Thời gian thực

Game, robot, drone: quyết định nhiều lần mỗi giây

Agent trình duyệt, máy tính

Chọn nút bấm và thao tác tiếp theo

Nền tảng cho agent

Định tuyến model, gác cổng tool, nén context

Phân loại khối lượng lớn

Ticket, email, tài liệu, log với giá vài xu

RAG và kiểm duyệt

Lọc đoạn văn liên quan, guardrail cho đầu ra

Điểm chung

Đáp án biết trước, và phải quyết định rất nhiều lần

PHẦN 2 · PHÂN LOẠI KHỐI LƯỢNG LỚN

Phân loại hàng nghìn bản ghi với giá vài xu

1.018 bài báo AI

8 cent

Trung vị 256 ms mỗi bài, chọn 1 trong 24 chủ đề. Model rẻ tóm tắt trước, Jev gán nhãn sau.

500 email

< 4 cent

Chia vào reply / research / wait / review, rồi mới giao cho agent soạn thư hoặc agent tìm hiểu.

3.282 bài trên X

12 cent

Phân loại xong trong 8 phút. Cùng kiểu với hóa đơn, ticket tồn đọng, dữ liệu bán hàng.

Công thức chung: Jev gán nhãn và độ tin cậy cho mọi bản ghi, chỉ những ca đáng giá nhất mới được chuyển tới agent đắt tiền.

Số liệu do tác giả các dự án tự công bố (1kpapers.com, Riley Brown, Chris Dunlop)

Model nhanh phản xạ, model chậm lên kế hoạch

Định tuyến model

LangChain · ModelRouterMiddleware

Việc dễ đi model rẻ, việc khó mới lên frontier. Một quyết định gần như miễn phí chặn được lời gọi đắt hơn nhiều.

Gác cổng tool

LangChain · AutoModeMiddleware

Trước mỗi lệnh shell: chỉ đọc, đảo ngược được hay phá hủy? Chắc thì chạy, nghi thì dừng hỏi người.

Nén context

fast-jev-compaction

Chấm điểm từng tool call, xóa phần cũ hết liên quan thay vì tóm tắt. Gần 1 triệu token còn 86K trong khoảng 1 giây.

Kiểm soát vòng lặp

Noul ở mọi bước

Xong chưa? Có tiến triển không? Có bị kẹt không? Đủ rẻ để hỏi liên tục thay vì chỉ kiểm tra khi đã hỏng.

PHẦN 2 · GỢI Ý CHO VIỆT NAM

Áp dụng ở ngân hàng và doanh nghiệp Việt

Chatbot ngân hàng

Nhận diện ý định kèm ngưỡng theo rủi ro: xem số dư thì ngưỡng thấp; chuyển tiền phải rất chắc mới làm, còn lại hỏi lại khách.

Chat với dữ liệu (BI)

Jev đoán trước câu hỏi chạm bảng, chủ đề nào; LLM viết SQL sau, đỡ dò cả từ điển dữ liệu. Ý tưởng đang được thử ở nước ngoài.

Khiếu nại, email, ticket

Phân bộ phận, chấm mức nghiêm trọng, gắn cờ gấp, tất cả trong một lần gọi.

RAG nội bộ và guardrail

Lọc đoạn văn trước khi đưa LLM; kiểm tra jailbreak, rò rỉ dữ liệu cá nhân ở đầu ra chatbot.

Jev và LLM: hai công cụ cho hai việc khác nhau

Tiêu chí	LLM hiện nay	Jev (System One)
Huấn luyện	RLHF / RLVR: theo sở thích người chấm	RLCD: xác suất khớp kết quả thực
Đầu ra	Chuỗi văn bản, phải parse, validate	Giá trị có kiểu, định nghĩa trước
Cách sinh	Tuần tự, từng token một	Song song, một lượt cho mọi câu hỏi
Giá	0,2–10 USD/1M token vào; ra đắt ~5 lần	0,042 USD/1M token vào; ra miễn phí
Tốc độ	3–329 giây (model frontier)	70–500 ms
Độ tin cậy	Hay tự tin quá mức, thiếu nhất quán	Xác suất hiệu chỉnh, nhất quán hơn
Hợp với	Chat, copilot, viết, code, suy luận	Workflow, “if thông minh”, real-time

Jev không thay GPT hay Claude cho việc viết và suy luận. Nó **thay cách làm phổ biến hiện nay: dùng một LLM rẻ để phân loại**, rồi viết thêm hàng trăm dòng code để bóc tách và kiểm tra câu trả lời.

PHẦN 3 · VÌ SAO NHANH

Một lượt tính thay vì T lượt nối đuôi nhau

LLM sinh một JSON dài 60 token

lượt 1 → lượt 2 → lượt 3 → · · · → lượt 60

Mỗi token cần một lượt forward pass riêng; lượt 40 phải đợi lượt 39. Thêm câu hỏi là thêm token, thêm thời gian.

Jev trả lời mọi câu hỏi cùng lúc

State + 3 câu hỏi → **1 lượt** → technical · 1.035 · 0.999

Trạng thái encode một lần, các câu hỏi đánh giá song song trên đó. Hỏi 10 câu tốn gần bằng thời gian hỏi 1 câu.

Khác với structured output: LLM vẫn sinh từng token, schema chỉ ràng buộc hình dạng chứ không bỏ được vòng lặp.

PHẦN 3 · CHUYỆN “KHÔNG ẢO GIÁC”

“Jev không thể ảo giác”: phải hiểu cho đúng

Được đảm bảo

Không bao giờ trả giá trị ngoài schema: không JSON hỏng, không khóa sai, không “bộ phận thứ tư”. Đảm bảo bằng cấu trúc, **0% lỗi kiểu**.

Không đảm bảo

Chọn **đúng** phương án 100%. Tất cả dựa vào kiến thức của model.

Ví dụ: tin nhắn “Tôi không đăng nhập được app”, phiếu chỉ có billing / technical / account. Jev không bao giờ trả lời “marketing” vì không có trong phiếu. Nhưng nó vẫn có thể chọn nhầm billing thay vì account.

Không lỗi kiểu ≠ không sai

Bài toán 1 triệu ticket: để Jev sàng lọc trước

Tất cả qua LLM mạnh (GPT, Claude...)

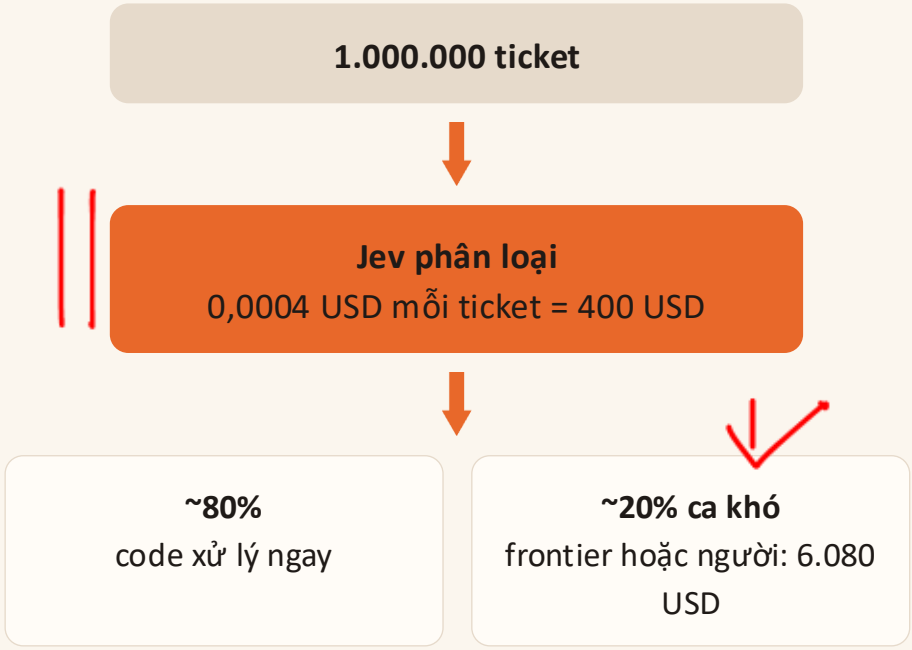
30.400 USD

khoảng 3 cent mỗi ticket, ~10 giây mỗi câu trả lời

Jev sàng lọc, chỉ ca khó mới lên model đắt

6.480 USD

và 800.000 ticket được trả lời trong chưa tới nửa giây



Phép tính trong bài "The Ultimate Guide to Jev". TypeSafe tự công bố: nhanh hơn 193,6 lần, rẻ hơn 444,6 lần (mức cao)

PHẦN 4 · GIỚI HẠN

Khi nào KHÔNG nên dùng Jev

Dữ liệu là do người viết, mỗi người nói một kiểu: “không vào được app”, “quên mật khẩu rồi”... đều là lỗi tài khoản

→ Jev

Đáp án mở: viết, tóm tắt, sinh code, giải thích

→ LLM

Dữ liệu đã là con số hoặc trường rõ ràng: số tiền trên 500 triệu, khách hàng loại VIP

→ một câu if là đủ

Điểm yếu đã được công bố

- **Làm toán, đếm, so sánh ngày tháng:** để code tính trước rồi mới hỏi Jev.
- **Suy luận qua nhiều bước:** tách thành nhiều câu hỏi nhỏ, hoặc dùng LLM.
- **Tự tìm thông tin chưa biết** (vd số hợp đồng trong văn bản): tìm các ứng viên bằng cách khác, rồi để Jev chọn.
- **Dữ liệu thừa:** gửi càng nhiều thứ không liên quan, Jev càng dễ sai.
- **Hiểu đúng từng chữ:** Jev không đoán ý bạn, nên câu hỏi phải viết thật rõ.
- **Dễ bị “dắt mũi”:** tin nhắn cố tình gợi ý đáp án có thể làm Jev chọn theo.
- **Giới hạn:** chỉ đọc chữ, tối đa 64K token, Choice tối đa 255 phương án.

Theo trang “jaggedness” và tài liệu giới hạn của Jev 1.13

Bắt đầu với Jev bản SaaS (có trả phí)

1

Đăng ký

typesafe.ai

Đang mở early access

2

Lấy API key

console.
typesafe.ai

Đặt vào TYPESAFE_API_KEY

3

Cài SDK

pip install
typesafe-sdk

@typesafe-ai/sdk

4

Gọi API

POST
/v1/systemone

jev-1.13.0

Kênh khác

OpenRouter: typesafe/jev-1.13, chuyển thẳng tới TypeSafe, không cộng phí

Vercel AI Gateway: typesafe-ai/jev qua AI SDK

Giá và giới hạn

0,042 USD / 1 triệu token vào, đầu ra 0 USD

Tổng 64K token; 32K cho state + câu hỏi dài nhất

Thông tin tại thời điểm cuối 9/2026, có thể thay đổi; kiểm tra lại trên typesafe.ai

OpenJev là một “họ” dự án cộng đồng

Dự án	Cách làm	Đáng chú ý
AlexWortega/openjev Hugging Face	Qwen3.5 thành cross-encoder NLI: đúng / mâu thuẫn / trung lập	MIT · 4B v2 đọc được ảnh, 4B v1, 35B MoE (3B active)
Zefan-Cai/Open-Jev GitHub	Context + câu hỏi + ứng viên → xác suất có kiểu	Bản 2B, 9B, 27B v1.1 · có workbench không cần code
SiliconLabAI/OpenJev GitHub	LLM chat thường đóng vai các bộ chấm điểm song song	620 ms, 9 lời gọi trên gpt-4o-mini · độ tin cậy chưa hiệu chỉnh
hr98w/jev-visual GitHub	Qwen3.5-0.8B chạy bằng MLX trên Mac, nhìn ảnh để chọn	Thử nghiệm, phục vụ học cơ chế

Nguồn: Hugging Face, GitHub, juliangoldie.com, apidog.com, news.qq.com (truy cập 10/2026)

Tự làm một “Jev mini” trong vài chục dòng code

Thực hành với Python.

https://github.com/mizorewww/laya-mlx

TỔNG KẾT

Thank you

1

Phần lớn lời gọi AI trong phần mềm là để **phán đoán**, không phải để viết.

2

LLM lo suy nghĩ, **Jev** lo quyết định, **code** lo thực thi.

3

Bắt đầu từ **một câu if** đang tốn tiền gọi LLM: đo, đặt ngưỡng, rồi mới mở rộng.